Unit IV – Inheritance and Pointers: Answers

Corrected code only

Section A – Debugging Questions

Question 1: Diamond Problem

```
#include <iostream>
class Person {
public:
    int idNumber;
    Person(int id = 0) : idNumber(id) {
        std::cout << "Person initialized: " << idNumber << std::endl;
    }
};
class Student : virtual public Person {
public:
    Student(int id) : Person(id) {}
};
class Employee : virtual public Person {
public:
    Employee(int id) : Person(id) {}
};
class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(int id)
        : Person(id), Student(id), Employee(id) {}
};
int main() {
    TeachingAssistant ta(101);
    std::cout << "TA ID: " << ta.idNumber << std::endl;
    return 0;
}
```

Question 2: Missing Base Constructor in Initializer List

```
#include <iostream>
#include <string>
class Device {
protected:
    std::string macAddress;
public:
    Device(std::string mac) : macAddress(mac) {
        std::cout << "Device configured with MAC: " << macAddress << std::endl;
    }
};
class Sensor : public Device {
private:
    int samplingRate;
public:
    Sensor(std::string mac, int rate) : Device(mac), samplingRate(rate) {
        std::cout << "Sensor constructed" << std::endl;
    }
    void display() const {
        std::cout << "MAC: " << macAddress << ", Rate: " << samplingRate << " Hz" << std::endl;
    }
};
int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

Question 3: Base Pointer Member Access & Object Slicing

```
#include <iostream>
class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() {}                       // needed for safe delete via base pointer
    void honk() { std::cout << "General Horn" << std::endl; }
};
class SportsCar : public Vehicle {
public:
    bool turboEnabled;
    SportsCar(int s, bool turbo) : Vehicle(s), turboEnabled(turbo) {}
    void engageTurbo() { std::cout << "Turbo Engaged!" << std::endl; }
};
int main() {
    Vehicle *vPtr = new SportsCar(300, true);
    vPtr->honk();
    static_cast<SportsCar*>(vPtr)->engageTurbo();   // explicit downcast
    SportsCar car(250, true);
    Vehicle &vRef = car;                            // reference: no slicing
    vRef.honk();
    std::cout << "Speed via reference: " << vRef.maxSpeed << std::endl;
    Vehicle sliced = car;                           // slicing: turboEnabled lost
    std::cout << "Sliced copy speed: " << sliced.maxSpeed << std::endl;
    delete vPtr;
    return 0;
}
```

Question 4: Method Chaining Return Types

```
#include <iostream>
class WindowBox {
private:
    int width, height;
public:
    WindowBox() : width(0), height(0) {}
    WindowBox& setWidth(int width) {
        this->width = width;
        return *this;
    }
    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }
    void display() const { std::cout << "Window: " << width << " x " << height << std::endl; }
};
int main() {
    WindowBox box;
    box.setWidth(800).setHeight(600);
    box.display();
    return 0;
}
```

Question 5: Private Inheritance

```
#include <iostream>
class RawSocket {
public:
    void openPort(int port) { std::cout << "Port " << port << " opened." << std::endl; }
    void closePort() { std::cout << "Port closed." << std::endl; }
};
class SecureChannel : private RawSocket {
public:
```

```
    using RawSocket::closePort;               // Approach A: access declaration
    void establishEncryptedTunnel(int port) {
        openPort(port);
        std::cout << "Tunnel encrypted with TLS 1.3" << std::endl;
    }
};
int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);
    channel.closePort();
    // RawSocket *socketPtr = &channel;   // removed: inaccessible base
    return 0;
}
```

Section B – Pseudo-Code Questions

Question 1: University ERP – Hybrid Inheritance with Virtual Base Classes

```
CLASS Person
  PROTECTED:
    STRING personID, fullName, email
  PUBLIC:
    CONSTRUCTOR Person(STRING id, STRING name, STRING mail)
      SET personID = id
      SET fullName = name
      SET email = mail
    ENDCONSTRUCTOR
ENDCLASS

CLASS Student VIRTUAL INHERITS Person
  PROTECTED:
    REAL gpa
    INTEGER creditsEarned
  PUBLIC:
    CONSTRUCTOR Student(STRING id, STRING name, STRING mail, REAL g, INTEGER credits)
        : Person(id, name, mail)           // ignored when a TeachingAssistant is built
      SET gpa = g
      SET creditsEarned = credits
    ENDCONSTRUCTOR
ENDCLASS

CLASS Faculty VIRTUAL INHERITS Person
  PROTECTED:
    STRING academicDepartment, rankTitle
  PUBLIC:
    CONSTRUCTOR Faculty(STRING id, STRING name, STRING mail, STRING dept, STRING rank)
        : Person(id, name, mail)           // ignored when a TeachingAssistant is built
      SET academicDepartment = dept
      SET rankTitle = rank
    ENDCONSTRUCTOR
ENDCLASS

CLASS TeachingAssistant INHERITS Student, Faculty
  PRIVATE:
    REAL stipendAmount
    STRING assignedCourse
  PUBLIC:
    CONSTRUCTOR TeachingAssistant(STRING id, STRING name, STRING mail, REAL g,
                                  INTEGER credits, STRING dept, STRING rank,
                                  REAL stipend, STRING course)
        : Person(id, name, mail),                      // direct delegation to virtual base
          Student(id, name, mail, g, credits),
          Faculty(id, name, mail, dept, rank)
      SET stipendAmount = stipend
      SET assignedCourse = course
```

```
    ENDCONSTRUCTOR

    METHOD DisplayTARecord()
      DISPLAY "ID         : ", personID           // single Person, no ambiguity
      DISPLAY "Name       : ", fullName
      DISPLAY "GPA        : ", gpa
      DISPLAY "Department : ", academicDepartment
      DISPLAY "Stipend    : $", stipendAmount
      DISPLAY "Course     : ", assignedCourse
    ENDMETHOD
ENDCLASS

BEGIN Main
  TeachingAssistant ta("U1001", "Priya Nair", "priya@uni.edu", 3.8, 96,
                       "Computer Science", "Teaching Assistant", 1200.00, "CS101")
  ta.DisplayTARecord()
END
```

Question 2: Polymorphic Payroll with Abstract Base Class

```
CLASS Employee ABSTRACT
  PROTECTED:
    STRING employeeID, name
  PUBLIC:
    CONSTRUCTOR Employee(STRING id, STRING n)
      SET employeeID = id
      SET name = n
    ENDCONSTRUCTOR
    VIRTUAL DESTRUCTOR ~Employee()
    ABSTRACT METHOD CalculatePay() RETURNS REAL         // pure virtual
    METHOD DisplayHeader()
      DISPLAY employeeID, " - ", name
    ENDMETHOD
ENDCLASS

CLASS SalariedEmployee INHERITS Employee
  PRIVATE:
    REAL monthlySalary
  PUBLIC:
    CONSTRUCTOR SalariedEmployee(STRING id, STRING n, REAL salary) : Employee(id, n)
      SET monthlySalary = salary
    ENDCONSTRUCTOR
    METHOD CalculatePay() RETURNS REAL OVERRIDE
      RETURN monthlySalary
    ENDMETHOD
ENDCLASS

CLASS HourlyEmployee INHERITS Employee
  PRIVATE:
    REAL hourlyWage, hoursWorked
  PUBLIC:
    CONSTRUCTOR HourlyEmployee(STRING id, STRING n, REAL wage, REAL hours) : Employee(id, n)
      SET hourlyWage = wage
      SET hoursWorked = hours
    ENDCONSTRUCTOR
    METHOD CalculatePay() RETURNS REAL OVERRIDE
      IF hoursWorked <= 40 THEN
        RETURN hoursWorked * hourlyWage
      ELSE
        RETURN (40 * hourlyWage) + ((hoursWorked - 40) * hourlyWage * 1.5)
      ENDIF
    ENDMETHOD
ENDCLASS

BEGIN Main
  POINTER TO Employee staff = ALLOCATE ARRAY OF POINTER TO Employee [2]
  REAL total, pay
```

```
  INTEGER i
  SET total = 0
  SET staff[0] = NEW SalariedEmployee("E01", "Anita", 8500)
  SET staff[1] = NEW HourlyEmployee("E02", "Rahul", 50, 48)
  FOR i = 0 TO 1 STEP 1 DO
    staff[i]->DisplayHeader()
    SET pay = staff[i]->CalculatePay()      // late binding
    DISPLAY "Pay: $", pay
    SET total = total + pay
  ENDFOR
  DISPLAY "Total disbursement: $", total
  FOR i = 0 TO 1 STEP 1 DO
    DELETE staff[i]                         // virtual destructor
  ENDFOR
  DEALLOCATE staff
END
```

Question 3: Composite Canvas Engine & this-Chaining

```
CLASS Point2D
  PRIVATE:
    REAL x, y
  PUBLIC:
    CONSTRUCTOR Point2D(REAL px = 0.0, REAL py = 0.0)
      SET x = px
      SET y = py
    ENDCONSTRUCTOR
    METHOD Set(REAL px, REAL py)
      SET x = px
      SET y = py
    ENDMETHOD
    METHOD GetX() RETURNS REAL
      RETURN x
    ENDMETHOD
    METHOD GetY() RETURNS REAL
      RETURN y
    ENDMETHOD
ENDCLASS

CLASS Circle
  PRIVATE:
    Point2D centerPoint            // composition: Circle HAS-A Point2D
    REAL radius
  PUBLIC:
    CONSTRUCTOR Circle() : centerPoint(0.0, 0.0)
      SET radius = 1.0
    ENDCONSTRUCTOR

    METHOD SetCenter(REAL newX, REAL newY) RETURNS REF Circle
      centerPoint.Set(newX, newY)
      RETURN REF THIS
    ENDMETHOD

    METHOD SetRadius(REAL r) RETURNS REF Circle
      IF r > 0 THEN
        SET radius = r
      ELSE
        DISPLAY "Invalid radius ignored"
      ENDIF
      RETURN REF THIS
    ENDMETHOD

    METHOD Draw()
      DISPLAY "Circle center=(", centerPoint.GetX(), ", ", centerPoint.GetY(),
              ") radius=", radius
    ENDMETHOD
ENDCLASS
```

```
BEGIN Main
  Circle myCircle
  myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()
END
```

Question 4: Multilevel Inheritance Fleet Telemetry

```
CLASS Vehicle
  PROTECTED:
    STRING vehicleVIN
    REAL baseWeightKg
  PUBLIC:
    CONSTRUCTOR Vehicle(STRING vin, REAL weight)
      SET vehicleVIN = vin
      SET baseWeightKg = weight
      DISPLAY "Vehicle constructed"
    ENDCONSTRUCTOR
    VIRTUAL DESTRUCTOR ~Vehicle()
      DISPLAY "Vehicle destroyed"
    ENDDESTRUCTOR
    METHOD DisplaySpecs()
      DISPLAY "VIN: ", vehicleVIN, "  Weight: ", baseWeightKg, " kg"
    ENDMETHOD
ENDCLASS

CLASS AutonomousVehicle INHERITS Vehicle
  PROTECTED:
    STRING lidarSensorID
    STRING firmwareVersion
  PUBLIC:
    CONSTRUCTOR AutonomousVehicle(STRING vin, REAL weight, STRING lidar, STRING fw)
        : Vehicle(vin, weight)
      SET lidarSensorID = lidar
      SET firmwareVersion = fw
      DISPLAY "AutonomousVehicle constructed"
    ENDCONSTRUCTOR
    DESTRUCTOR ~AutonomousVehicle()
      DISPLAY "AutonomousVehicle destroyed"
    ENDDESTRUCTOR
ENDCLASS

CLASS DeliveryDrone INHERITS AutonomousVehicle
  PRIVATE:
    REAL payloadCapacityKg
    REAL batteryChargePercent
  PUBLIC:
    CONSTRUCTOR DeliveryDrone(STRING vin, REAL weight, STRING lidar, STRING fw,
                              REAL payload, REAL battery)
        : AutonomousVehicle(vin, weight, lidar, fw)
      SET payloadCapacityKg = payload
      SET batteryChargePercent = battery
      DISPLAY "DeliveryDrone constructed"
    ENDCONSTRUCTOR
    DESTRUCTOR ~DeliveryDrone()
      DISPLAY "DeliveryDrone destroyed"
    ENDDESTRUCTOR
    METHOD DisplayFlightStatus()
      DisplaySpecs()
      DISPLAY "LiDAR: ", lidarSensorID, "  Firmware: ", firmwareVersion
      DISPLAY "Payload: ", payloadCapacityKg, " kg  Battery: ", batteryChargePercent, "%"
    ENDMETHOD
ENDCLASS

BEGIN Main
  POINTER TO DeliveryDrone dronePtr = NEW DeliveryDrone("VIN123", 5.5, "LD-77", "v2.1", 3.0,
85.0)
```

```
  POINTER TO Vehicle basePtr = dronePtr         // implicit upcast
  basePtr->DisplaySpecs()                       // only Vehicle members visible
  POINTER TO DeliveryDrone d2 = CAST basePtr TO POINTER TO DeliveryDrone   // explicit downcast
  d2->DisplayFlightStatus()
  DELETE basePtr
END
```

Question 5: Pointer-Swap Bubble Sort of Products

```
CLASS Product
  PRIVATE:
    STRING sku
    REAL price
    INTEGER stockCount
  PUBLIC:
    CONSTRUCTOR Product(STRING s, REAL p, INTEGER stock)
      SET sku = s
      SET price = p
      SET stockCount = stock
    ENDCONSTRUCTOR
    METHOD GetPrice() RETURNS REAL
      RETURN price
    ENDMETHOD
    METHOD PrintProduct()
      DISPLAY sku, "  $", price, "  stock: ", stockCount
    ENDMETHOD
ENDCLASS

PROCEDURE SortCatalog(ARRAY catalog OF POINTER TO Product, INTEGER N)
  INTEGER pass, j
  POINTER TO Product temp
  FOR pass = 0 TO N - 2 STEP 1 DO
    FOR j = 0 TO N - 2 - pass STEP 1 DO
      IF catalog[j]->GetPrice() > catalog[j+1]->GetPrice() THEN
        SET temp = catalog[j]                 // swap POINTERS only
        SET catalog[j] = catalog[j+1]
        SET catalog[j+1] = temp
      ENDIF
    ENDFOR
  ENDFOR
ENDPROCEDURE

BEGIN Main
  INTEGER N = 3, i
  ARRAY catalog[3] OF POINTER TO Product
  SET catalog[0] = NEW Product("Laptop",  1200, 10)
  SET catalog[1] = NEW Product("Mouse",     25, 150)
  SET catalog[2] = NEW Product("Monitor",  350, 40)
  SortCatalog(catalog, N)
  FOR i = 0 TO N - 1 STEP 1 DO
    catalog[i]->PrintProduct()
  ENDFOR
  FOR i = 0 TO N - 1 STEP 1 DO
    DELETE catalog[i]
  ENDFOR
END
```